



MVIDCodeReader SDK (C)

Developer Guide

Legal Information

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE DOCUMENT IS PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". OUR COMPANY MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT. IN NO EVENT WILL OUR COMPANY BE LIABLE FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, IN CONNECTION WITH THE USE OF THE DOCUMENT, EVEN IF OUR COMPANY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.

Contents

Chapter 1 Overview	1
1.1 Introduction	1
1.2 Development Environment	1
1.3 Update History	1
Chapter 2 Typical Applications	2
2.1 Acquire Camera Image and Read Barcode	2
2.2 Upload Local Image and Read Barcode	4
Chapter 3 API Reference	5
3.1 Common APIs	5
3.1.1 MVID_CR_GetVersion	5
3.1.2 MVID_CR_CreateHandle	5
3.1.3 MVID_CR_DestroyHandle	6
3.2 Camera Settings APIs	6
3.2.1 MVID_CR_CAM_EnumDevices	6
3.2.2 MVID_CR_CAM_EnumDevicesByCfg	7
3.2.3 MVID_CR_CAM_BindDevice	7
3.2.4 MVID_CR_CAM_BindDeviceByIP	7
3.2.5 MVID_CR_CAM_BindDeviceBySerialNumber	8
3.2.6 MVID_CR_CAM_SetIntValue	9
3.2.7 MVID_CR_CAM_GetIntValue	9
3.2.8 MVID_CR_CAM_SetFloatValue	10
3.2.9 MVID_CR_CAM_GetFloatValue	10
3.2.10 MVID_CR_CAM_SetBoolValue	11
3.2.11 MVID_CR_CAM_GetBoolValue	11
3.2.12 MVID_CR_CAM_SetEnumValue	12
3.2.13 MVID_CR_CAM_SetEnumValueByString	13

3.2.14 MVID_CR_CAM_GetEnumValue	13
3.2.15 MVID_CR_CAM_SetStringValue	14
3.2.16 MVID_CR_CAM_GetStringValue	14
3.2.17 MVID_CR_CAM_SetCommandValue	15
3.3 Functional APIs	15
3.3.1 MVID_CR_Process	15
3.3.2 MVID_CR_CAM_RegisterImageCallBack	16
3.3.3 MVID_CR_CAM_RegisterImageBufferCallBack	17
3.3.4 MVID_CR_CAM_RegisterAllEventCallBack	18
3.3.5 MVID_CR_StartGrabbing	19
3.3.6 MVID_CR_CAM_SetImageNodeNum	19
3.3.7 MVID_CR_CAM_SetImageOutPutMode	20
3.3.8 MVID_CR_CAM_GetOneFrameTimeout	20
3.3.9 MVID_CR_CAM_GetImageBuffer	21
3.3.10 MVID_CR_CAM_RegisterPreImageCallBack	21
3.3.11 MVID_CR_RegisterExceptionCallBack	22
3.3.12 MVID_CR_SaveImage	23
3.3.13 MVID_CR_GetImageFileData	24
3.3.14 MVID_CR_RuleLoad	24
3.3.15 MVID_CR_ScriptLoad	25
3.3.16 MVID_CR_StopGrabbing	26
3.4 Algorithm Settings APIs	26
3.4.1 MVID_CR_Algorithm_SetIntValue	26
3.4.2 MVID_CR_Algorithm_GetIntValue	27
3.4.3 MVID_CR_Algorithm_SetFloatValue	27
3.4.4 MVID_CR_Algorithm_GetFloatValue	28
3.4.5 MVID_CR_Algorithm_SetStringValue	28
3.4.6 MVID_CR_Algorithm_GetStringValue	29

Chapter 4 Data Structure and Enumeration	31
4.1 Data Structure	31
4.1.1 MVID_CAM_ENUMVALUE	31
4.1.2 MVID_CAM_FLOATVALUE	31
4.1.3 MVID_CAM_INTVALUE_EX	31
4.1.4 MVID_CAM_OUTPUT_INFO	32
4.1.5 MVID_CAM_STRINGVALUE	32
4.1.6 MVID_CAMERA_INFO	32
4.1.7 MVID_CAMERA_INFO_LIST	33
4.1.8 MVID_CODE_INFO	34
4.1.9 MVID_CODE_INFO_LIST	34
4.1.10 MVID_EVENT_OUT_INFO	34
4.1.11 MVID_IMAGE_INFO	35
4.1.12 MVID_IMAGE_OUTPUT_MODE	35
4.1.13 MVID_POINT_I	36
4.1.14 MVID_PROC_PARAM	36
4.2 Enumeration	37
4.2.1 MVID_CODE_TYPE	37
4.2.2 MVID_IMAGE_TYPE	37
4.2.3 MVID_IMAGE_OUTPUT_MODE	39
Appendix A. Appendixes	40
A.1 Algorithm Parameters	40
A.2 Error Code	44

Chapter 1 Overview

1.1 Introduction

The MVIDCodeReader SDK is a software development kit using C language, which can cooperate with industrial cameras to acquire images with barcode information, recognize 1D barcodes or 2D barcodes, set parameters, filter barcodes, and so on.

1.2 Development Environment

The development environment of MVIDCodeReader SDK is shown in the table below.

Item	Required
CPU	Intel Pentium IV 3.0 GHZ and above
RAM	1 GB and above
Operating System	Microsoft® Windows XP (32-bit)/Windows 7 (32/64-bit)/Windows 10 (32/64-bit)
MVS	V3.0.0 and above



Note

- For the operating system of Microsoft Windows XP, you should install the Microsoft .NET Framework 3.5 Service Pack 1.
 - When installing dongle, you should insert iMVS1001, or iMVS6000 (CodeReader).
 - Support soft encryption, and certain types of cameras can be unencrypted.
-

1.3 Update History

Summary of Changes in Version 1.2.0_Apr. 2023

Added an API for reading and converting the image: ***MVID_CR_GetImageFileData***

Summary of Changes in Version 1.1.0_Oct. 2019

New document.

Chapter 2 Typical Applications

2.1 Acquire Camera Image and Read Barcode

Steps

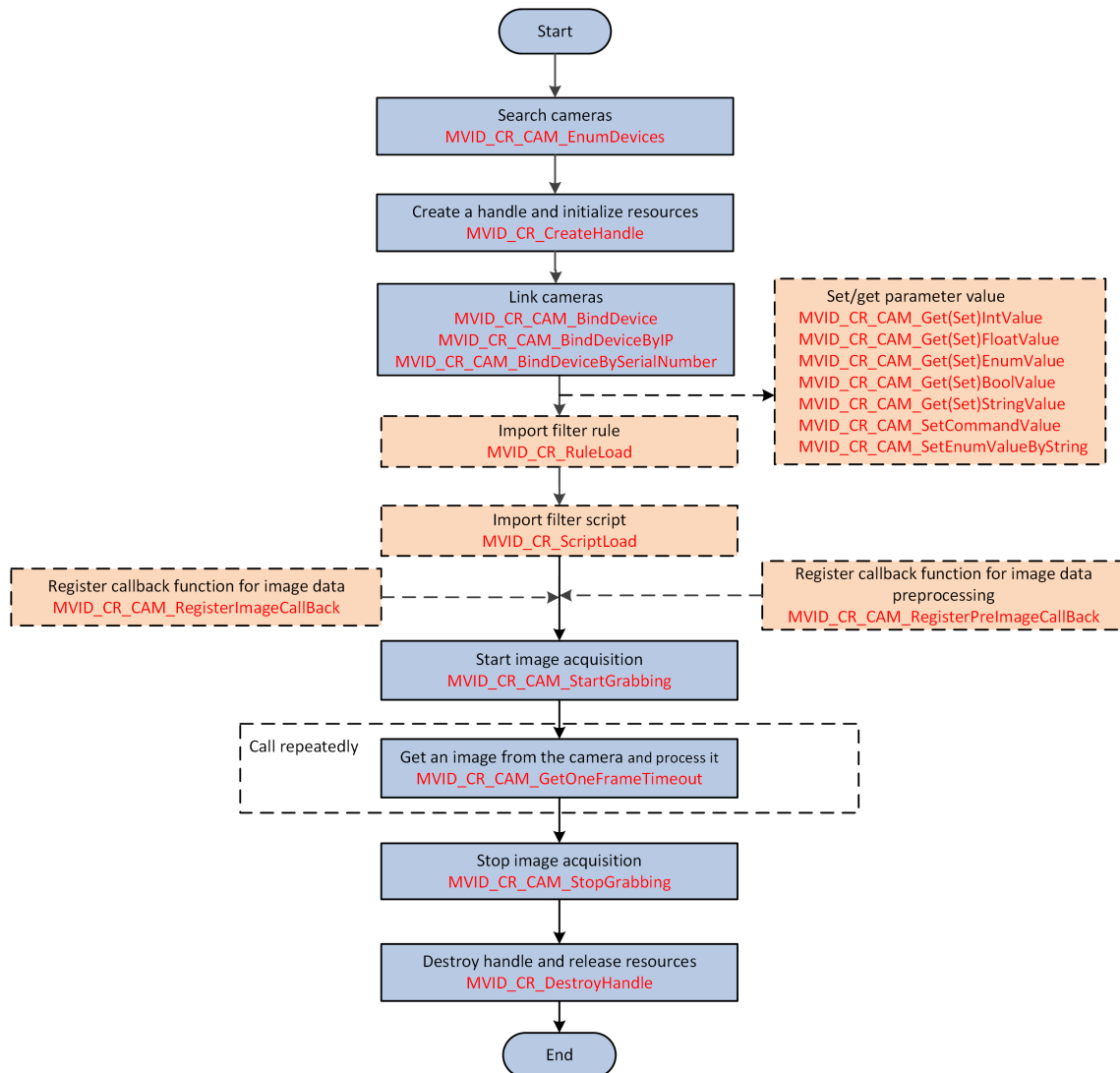


Figure 2-1 Programming Flow of Acquiring Camera Image and Reading Barcode

1. Call **MVID_CR_CAM_EnumDevices** to search for devices.
2. Call **MVID_CR_CreateHandle** to create the handle.
3. Link cameras.
 - Call **MVID_CR_CAM_BindDevice** to link cameras via camera information.
 - Call **MVID_CR_CAM_BindDeviceByIP** to link cameras via IP address.
 - Call **MVID_CR_CAM_BindDeviceBySerialNumber** to link cameras via serial No.

4. Optional: Perform the following operation(s) to get or set different types of parameter values.

Parameters of type int	Call <u>MVID_CR_CAM_GetIntValue</u> / <u>MVID_CR_CAM_SetIntValue</u>
Parameters of type float	Call <u>MVID_CR_CAM_GetFloatValue</u> / <u>MVID_CR_CAM_SetFloatValue</u>
Parameters of type enum	Call <u>MVID_CR_CAM_GetEnumValue</u> / <u>MVID_CR_CAM_SetEnumValue</u> / <u>MVID_CR_CAM_SetEnumValueByString</u>
Parameters of type boolean	Call <u>MVID_CR_CAM_GetBoolValue</u> / <u>MVID_CR_CAM_SetBoolValue</u>
Parameters of type string	Call <u>MVID_CR_CAM_GetStringValue</u> / <u>MVID_CR_CAM_SetStringValue</u>
Parameters of type command	Call <u>MVID_CR_CAM_SetCommandValue</u>

5. Optional: Call **MVID_CR_RuleLoad** to import filtering rule files.

6. Optional: Call **MVID_CR_ScriptLoad** to import filtering script files.

7. Optional: Call **MVID_CR_CAM_RegisterPreImageCallBack** to register callback function for preprocessing image data.

8. Optional: Call **MVID_CR_CAM_RegisterImageCallBack** to register callback function for getting image data.

9. Call **MVID_CR_StartGrabbing** to start image acquisition.

10. Call **MVID_CR_CAM_GetOneFrameTimeout** repeatedly to get images from the camera.



Note

The API is invalid after calling **MVID_CR_CAM_RegisterImageCallBack** . You should get the image data from the registered callback function (cbOutput).

11. Call **MVID_CR_StopGrabbing** to stop image acquisition.

12. Call **MVID_CR_DestroyHandle** to destroy the handle.

2.2 Upload Local Image and Read Barcode

Steps

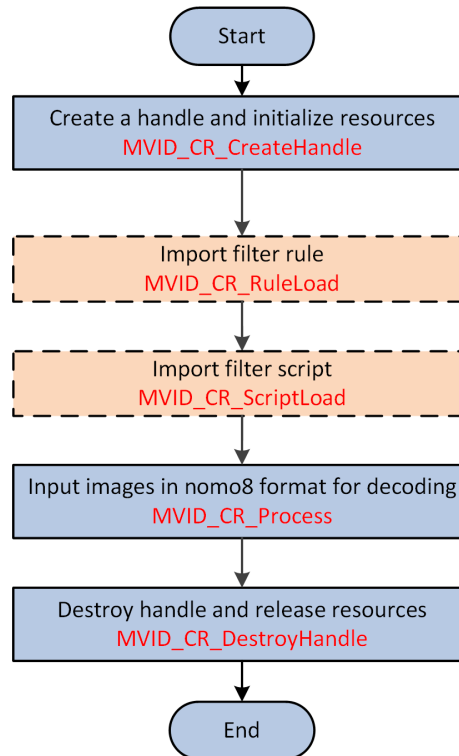


Figure 2-2 Programming Flow of Uploading Local Image and Reading Barcode

1. Call **MVID_CR_CreateHandle** to create the handle.
2. **Optional:** Call **MVID_CR_RuleLoad** to import filtering rule files.
3. **Optional:** Call **MVID_CR_ScriptLoad** to import filtering script files.
4. Call **MVID_CR_Process** to upload images in mono8 format for decoding.
5. Call **MVID_CR_DestroyHandle** to destroy the handle.

Chapter 3 API Reference

3.1 Common APIs

3.1.1 MVID_CR_GetVersion

Get version No. of SDK, 1D barcode, or 2D barcode.

API Definition

```
int MVID_CR_GetVersion(  
    char*const      chVersion  
);
```

Parameters

chVersion

[IN][OUT] Version No.

Return Value

Return version No. of SDK, 1D barcode, or 2D barcode for success. For example, 1.2.0.1_3.6.2_2.2.5.

3.1.2 MVID_CR_CreateHandle

Create a handle.

API Definition

```
int MVID_CR_CreateHandle(  
    void            **handle,  
    unsigned int    nCodeAbility  
);
```

Parameters

handle

[OUT] Device handle.

nCodeAbility

[IN] The capability set to be used. For example, you can call the command "MVID_BCR|MVID_TDCR" to enable functions of 1D barcode and 2D barcode.

Return Value

Return *0* for success, and return ***Error Code*** for failure.

Remarks

Do not connect the camera when creating a handle by this API.

3.1.3 MVID_CR_DestroyHandle

Destroy handle and release resources.

API Definition

```
int MVID_CR_DestroyHandle(  
    void *handle  
);
```

Parameters

handle

[IN] Handle, which is returned by ***MVID_CR_CreateHandle***.

Remarks

Return *0* for success, and return ***Error Code*** for failure.

3.2 Camera Settings APIs

3.2.1 MVID_CR_CAM_EnumDevices

Search for cameras on the current LAN.

API Definition

```
int MVID_CR_CAM_EnumDevices(  
    MVID_CAMERA_INFO_LIST *pstCamList  
);
```

Parameters

pstCamList

[IN][OUT] Camera information list. See ***MVID_CAMERA_INFO_LIST*** for details.

Return Value

Return *0* for success, and return ***Error Code*** for failure.

3.2.2 MVID_CR_CAM_EnumDevicesByCfg

Search for specific cameras by configuration files.

API Definition

```
int MVID_CR_CAM_EnumDevicesByCfg(  
    MVID_CAMERA_INFO_LIST    *pstCamList  
);
```

Parameters

pstCamList

[IN][OUT] Camera information list. See [**MVID_CAMERA_INFO_LIST**](#) for details.

Return Value

Return 0 for success, and return [**Error Code**](#) for failure.

3.2.3 MVID_CR_CAM_BindDevice

Link cameras.

API Definition

```
int MVID_CR_CAM_BindDevice(  
    void                *handle,  
    const MVID_CAMERA_INFO    *pstCamInfo  
);
```

Parameters

handle

[IN] Handle, which is returned by [**MVID_CR_CreateHandle**](#) .

pstCamInfo

[IN] Device information structure. See [**MVID_CAMERA_INFO**](#) for details.

Return Value

Return 0 for success, and return [**Error Code**](#) for failure.

3.2.4 MVID_CR_CAM_BindDeviceByIP

Link a camera by IP address.

API Definition

```
int MVID_CR_CAM_BindDeviceByIP(  
    void          *handle,  
    const char     *chCurrentIp,  
    const char     *chNetExport  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

chCurrentIp

[IN] Camera's IP address

chNetExport

[IN] Current PC's IP address.

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.2.5 MVID_CR_CAM_BindDeviceBySerialNumber

Link a camera by serial No.

API Definition

```
int MVID_CR_CAM_BindDeviceBySerialNumber(  
    void          *handle,  
    const char     *chSerialNumber  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

chSerialNumber

[IN] Camera serial No.

Return Value

Return 0 for success, and return [Error Code](#) for failure.

Reamrks

Only one camera can be bound via one handle.

3.2.6 MVID_CR_CAM_SetIntValue

Set parameters of type int.

API Definition

```
int MVID_CR_CAM_SetIntValue(  
    void                *handle,  
    const char          *strKey,  
    int64_t             nValue  
);
```

Parameters

handle

[IN] Handle, which is returned by **MVID_CR_CreateHandle** .

strKey

[IN] Parameter name

nValue

[IN] Parameter value

Return Value

Return 0 for success, and return **Error Code** for failure.

3.2.7 MVID_CR_CAM_GetIntValue

Get parameters of type int.

API Definition

```
int MVID_CR_CAM_GetIntValue(  
    void                *handle,  
    const char          *strKey,  
    MVID_CAM_INTVALUE_EX *pIntValue  
);
```

Parameters

handle

[IN] Handle, which is returned by **MVID_CR_CreateHandle** .

strKey

[IN] Parameter name

pIntValue

[OUT] Parameter value list, which contains the maximum values, minimum values, and current values of the parameters. See [**MVID_CAM_INTVALUE_EX**](#) for details.

Return Value

Return *0* for success, and return [**Error Code**](#) for failure.

3.2.8 MVID_CR_CAM_SetFloatValue

Set parameters of type float.

API Definition

```
int MVID_CR_CAM_SetFloatValue(  
    void                *handle,  
    const char          *strKey,  
    float               fValue  
);
```

Parameters

handle

[IN] Handle, which is returned by [**MVID_CR_CreateHandle**](#) .

strKey

[IN] Parameter name

fValue

[IN] Parameter value

Return Value

Return *0* for success, and return [**Error Code**](#) for failure.

3.2.9 MVID_CR_CAM_GetFloatValue

Get parameters of type float.

API Definition

```
int MVID_CR_CAM_GetFloatValue(  
    void                *handle,  
    const char          *strKey,  
    MVID_CAM_FLOATVALUE *pFloatValue  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

strKey

[IN] Parameter name

pFloatValue

[IN][OUT] Parameter value list. See [MVID_CAM_FLOATVALUE](#) for details.

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.2.10 MVID_CR_CAM_SetBoolValue

Set parameters of type boolean.

API Definition

```
int MVID_CR_CAM_SetBoolValue(  
    void          *handle,  
    const char    *strKey,  
    bool          bValue  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

strKey

[IN] Parameter name

bValue

[IN] Parameter value

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.2.11 MVID_CR_CAM_GetBoolValue

Get parameters of type boolean.

API Definition

```
int MVID_CR_CAM_GetBoolValue(  
    void            *handle,  
    const char      *strKey,  
    bool            *pBoolValue  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

strKey

[IN] Parameter name

pBoolValue

[IN][OUT] Parameter value

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.2.12 MVID_CR_CAM_SetEnumValue

Set parameters of type enum.

API Definition

```
int MVID_CR_CAM_SetEnumValue(  
    void            *handle,  
    const char      *strKey,  
    unsigned int     nValue  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

strKey

[IN] Parameter name

nValue

[IN] Parameter value

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.2.13 MVID_CR_CAM_SetEnumValueByString

Set parameters of type enum by string.

API Definition

```
int MVID_CR_CAM_SetEnumValueByString(  
    void          *handle,  
    const char    *strKey,  
    const char    *sValue  
);
```

Parameters

handle

[IN] Handle, which is returned by ***MVID_CR_CreateHandle*** .

strKey

[IN] Parameter name

sValue

[IN] Parameter value

Return Value

Return 0 for success, and return ***Error Code*** for failure.

Remarks

Machine vision cameras prior to V2.3.1 do not support the API.

3.2.14 MVID_CR_CAM_GetEnumValue

Get parameters of type enum.

API Definition

```
int MVID_CR_CAM_GetEnumValue(  
    void          *handle,  
    const char    *strKey,  
    MVID_CAM_ENUMVALUE *pEnumValue  
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name

pEnumValue

[IN][OUT] Parameter value list, which contains the current value and the number of valid values.
See [MVID_CAM_ENUMVALUE](#) for details.

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.2.15 MVID_CR_CAM_SetStringValue

Set parameters of type string.

API Definition

```
int MVID_CR_CAM_SetStringValue(  
    void                *handle,  
    const char          *strKey,  
    const char          *sValue  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

strKey

[IN] Parameter name

sValue

[IN] Parameter value

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.2.16 MVID_CR_CAM_GetStringValue

Get parameters of type string.

API Definition

```
int MVID_CR_CAM_GetStringValue(  
    void                *handle,  
    const char          *strKey,  
    MVID_CAM_STRINGVALUE *pStringValue  
);
```

Parameters

handle

[IN] Handle.

strKey

[IN] Parameter name

pStringValue

[IN][OUT] Parameter value. See [***MVID_CAM_STRINGVALUE***](#) for details.

Return Value

Return *0* for success, and return [***Error Code***](#) for failure.

3.2.17 MVID_CR_CAM_SetCommandValue

Set parameters of type command.

API Definition

```
int MVID_CR_CAM_SetCommandValue(  
    void          *handle,  
    const char     *strKey,  
);
```

Parameters

handle

[IN] Handle, which is returned by [***MVID_CR_CreateHandle***](#) .

strKey

[IN] Parameter name

Return Value

Return *0* for success, and return [***Error Code***](#) for failure.

3.3 Functional APIs

3.3.1 MVID_CR_Process

Read code.

API Definition

```
int MVID_CR_Process(  
    void *handle,  
    MVID_PROC_PARAM *pstParam,  
    unsigned int nCodeAbility  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

pstParam

[IN][OUT] Image information. See [MVID_PROC_PARAM](#) for details.

nCodeAbility

[IN] Code capability set

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.3.2 MVID_CR_CAM_RegisterImageCallBack

Register callback function of image data.

API Definition

```
int MVID_CR_CAM_RegisterImageCallBack(  
    void *handle,  
    void *cbOutput,  
    void *pUser  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

cbOutput

[IN] Image data callback function

```
void cbOutput(  
    MVID_CAM_OUTPUT_INFO *pstOutput,  
    void *pUser  
)
```

pstOutput

[OUT] Obtained image information. See [***MVID_CAM_OUTPUT_INFO***](#) for details.

pUser

[OUT] User pointer

pUser

[IN] User pointer

Return Value

Return 0 for success, and return [***Error Code***](#) for failure.

3.3.3 MVID_CR_CAM_RegisterImageBufferCallBack

Register callback function for getting an original image.

API Definition

```
int MVID_CR_CAM_RegisterImageBufferCallBack(  
void      *handle,  
void      *cbOutput,  
void      *pUser  
);
```

Parameters**handle**

[IN] Handle, which is returned by [***MVID_CR_CreateHandle***](#) .

cbOutput

[IN] Image data callback function

```
void cbOutput(  
MVID_CAM_OUTPUT_INFO  *pstOutput,  
void                  *pUser  
)
```

pstOutput

[OUT] Obtained image information. See [***MVID_CAM_OUTPUT_INFO***](#) for details.

pUser

[OUT] User pointer

pUser

[IN] User pointer

Return Value

Return *0* for success, and return ***Error Code*** for failure.

Remarks

The API is invalid after calling ***MVID_CR_CAM_RegisterImageCallBack***.

3.3.4 MVID_CR_CAM_RegisterAllEventCallBack

Register callback function for getting camera events.

API Definition

```
int MVID_CR_CAM_RegisterAllEventCallBack(  
    void      *handle,  
    void      *cbEvent,  
    void      *pUser  
);
```

Parameters

handle

[IN] Handle, which is returned by ***MVID_CR_CreateHandle***.

cbEvent

[IN] Callback function of camera events.

```
void cbEvent(  
    MVID_EVENT_OUT_INFO  *pEventInfo,  
    void                  *pUser  
)
```

nMsgType

[OUT] Event information. See ***MVID_EVENT_OUT_INFO*** for details.

pUser

[OUT] User pointer

pUser

[IN] User pointer

Return Value

Return *0* for success, and return ***Error Code*** for failure.

Remarks

The API should be supported by cameras.

3.3.5 MVID_CR_StartGrabbing

Start image acquisition.

API Definition

```
int MVID_CR_StartGrabbing(  
    void          *handle  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.3.6 MVID_CR_CAM_SetImageNodeNum

Set the number of image cache nodes.

API Definition

```
int MVID_CR_CAM_SetImageNodeNum(  
    void          *handle,  
    unsigned int   nNum  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

nNum

[IN] The number of cache nodes, which cannot be less than 1.

Return Value

Return 0 for success, and return [Error Code](#) for failure.

Remarks

The API should be called before acquiring images.

3.3.7 MVID_CR_CAM_SetImageOutPutMode

Set image output format.

API Definition

```
int MVID_CR_CAM_SetImageOutPutMode(  
    void *handle,  
    MVID_IMGAE_OUTPUT_MODE enImageOutPutMode  
);
```

Parameters

handle

[IN] Handle, which is returned by [***MVID_CR_CreateHandle***](#).

enImageOutPutMode

[IN] Image output format. See [***MVID_IMAGE_OUTPUT_MODE***](#) for details.

Return Value

Return 0 for success, and return [***Error Code***](#) for failure.

3.3.8 MVID_CR_CAM_GetOneFrameTimeout

Get one picture in a blocking mode and process it by algorithm.

API Definition

```
int MVID_CR_CAM_GetOneFrameTimeout(  
    void *handle,  
    MVID_CAM_OUTPUT_INFO *pstFrameInfo,  
    unsigned int nMsec  
);
```

Parameters

handle

[IN] Handle, which is returned by [***MVID_CR_CreateHandle***](#).

pstFrameInfo

[IN] [OUT] Image information. See [***MVID_CAM_OUTPUT_INFO***](#) for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired within the timeout.

Return Value

Return *0* for success, and return **Error Code** for failure.

3.3.9 MVID_CR_CAM_GetImageBuffer

Get one original picture from the camera in a blocking mode.

API Definition

```
int MVID_CR_CAM_GetImageBuffer(  
    void *handle,  
    MVID_IMAGE_INFO *pImageInfo,  
    unsigned int nMsec  
);
```

Parameters

handle

[IN] Handle, which is returned by **MVID_CR_CreateHandle** .

pImageInfo

[IN] [OUT] Image information. See **MVID_IMAGE_INFO** for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired when exceeding the timeout.

Return Value

Return *0* for success, and return **Error Code** for failure.

Remarks

The API is invalid after calling **MVID_CR_CAM_GetOneFrameTimeout** .

3.3.10 MVID_CR_CAM_RegisterPreImageCallBack

Register callback function for preprocessing image data .

API Definition

```
int MVID_CR_CAM_RegisterPreImageCallBack(  
    void *handle,  
    void *cbPreOutput,  
    void *pUser  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

cbPreOutput

[IN] Callback function of image data preprocessing

```
void cbPreOutput(  
    MVID_IMAGE_INFO    *pstPreOutput,  
    MVID_IMAGE_INFO    *pstProcInput,  
    void                *pUser  
)
```

pstPreOutput

[OUT] Output image

pstProcInput

[OUT] Input image

pUser

[OUT] User pointer

pUser

[IN] User pointer

Return Value

Return 0 for success, and return [Error Code](#) for failure.

Remarks

The format of input images should be mono8 after preprocessing.

3.3.11 MVID_CR_RegisterExceptionCallBack

Register callback function for getting exception information of cameras and dongles.

API Definition

```
int MVID_CR_RegisterExceptionCallBack(  
    void        *handle,  
    void        *cbException,  
    void        *pUser  
);
```

Parameters

handle

[IN] Handle, which is returned by **MVID_CR_CreateHandle** .

cbException

[IN] Exception information callback function

```
void cbException(  
    unsigned int    nMsgType,  
    void            *pUser  
)
```

nMsgType

[OUT] Exception message.

pUser

[OUT] User pointer

pUser

[IN] User pointer

Return Value

Return 0 for success, and return **Error Code** for failure.

Remarks

You can get exception information via this API without linking a camera.

3.3.12 MVID_CR_SaveImage

Save the image in BMP or JPEG format with the encoding quality between 50 and 99.

API Definition

```
int MVID_CR_SaveImage(  
    void            *handle,  
    MVID_IMAGE_INFO *pstInputImage,  
    MVID_IMAGE_TYPE  enImageType,  
    MVID_IMAGE_INFO *pstOutputImage,  
    unsigned int     nJpgQuality = 80  
);
```

Parameters

handle

[IN] Handle, which is returned by **MVID_CR_CreateHandle** .

pstInputImage

[IN] Input image parameters. See **MVID_IMAGE_INFO** for details.

enImageType

[IN] The format to convert to, which is JPEG by default. See **MVID_IMAGE_TYPE** for details.

nJpgQuality

[IN] JPG encoding quality , which is 80 by default. It is valid only when **enImageType** is "JPEG".

pstOutputImage

[OUT] Output image parameters. See **MVID_IMAGE_INFO** for details.

Return Value

Return 0 for success, and return **Error Code** for failure.

3.3.13 MVID_CR_GetImageFileData

Read the image and convert it to mono8 format, which can be used in MVID_CR_Process.

API Definition

```
int MVID_CR_GetImageFileData(
    void                *handle,
    const char          *pFilePath
    MVID_PROC_PARAM    *pstImageParam
);
```

Parameters

handle

[IN] Handle, which is returned by **MVID_CR_CreateHandle** .

pFilePath

[IN] Path of the input image file.

pstImageParam

[IN][OUT] Structure of the output image

Return Value

Return 0 for success, and return **Error Code** for failure.

Remarks

Supports images in RAW, BMP, JPEG, format. For RAW images, the input of width, height, and pixel format is required.

3.3.14 MVID_CR_RuleLoad

Import filter rule files.

API Definition

```
int MVID_CR_RuleLoad(  
    void                *handle,  
    const char          *pFilePath  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

pFilePath

[IN] Path of the filter rule file. You can set it to NULL to cancel the filter.

Return Value

Return 0 for success, and return [Error Code](#) for failure.

Remarks

Filter rule files are generated by the tool RuleFilter.

3.3.15 MVID_CR_ScriptLoad

Import script files.

API Definition

```
int MVID_CR_ScriptLoad(  
    void                *handle,  
    const char          *pFilePath,  
    const char          *pFuncName  
);
```

Parameters

handle

[IN] Handle, which is returned by [MVID_CR_CreateHandle](#) .

pFilePath

[IN] Path of the script file

pFuncName

[IN] Filter function name

Return Value

Return 0 for success, and return [Error Code](#) for failure.

3.3.16 MVID_CR_StopGrabbing

Stop image acquisition.

API Definition

```
int MVID_CR_StopGrabbing(  
    void          *handle  
);
```

Parameters

handle

[IN] Handle, which is returned by [***MVID_CR_CreateHandle***](#) .

Return Value

Return 0 for success, and return [***Error Code***](#) for failure.

3.4 Algorithm Settings APIs

3.4.1 MVID_CR_Algorithm_SetIntValue

Set algorithm parameters of type int.

API Definition

```
int MVID_CR_Algorithm_SetIntValue(  
    void          *handle,  
    const char* const strParamKeyName,  
    const int      nValue  
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to [***Algorithm Parameters***](#) .

nValue

[IN] Parameter value

Return Value

Return *0* for success, and return **Error Code** for failure.

3.4.2 MVID_CR_Algorithm_GetIntValue

Get algorithm parameters of type int.

API Definition

```
int MVID_CR_Algorithm_GetIntValue(  
    void *handle,  
    const char* const strParamKeyName,  
    int* const pnValue  
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to **Algorithm Parameters** .

pnValue

[IN][OUT] Parameter value

Return Value

Return *0* for success, and return **Error Code** for failure.

3.4.3 MVID_CR_Algorithm_SetFloatValue

Set algorithm parameters of type float.

API Definition

```
int MVID_CR_Algorithm_SetFloatValue(  
    void *handle,  
    const char* const strParamKeyName,  
    const float fValue  
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to **Algorithm Parameters** .

fValue

[IN] Parameter value

Return Value

Return 0 for success, and return **Error Code** for failure.

3.4.4 MVID_CR_Algorithm_GetFloatValue

Get algorithm parameters of type float.

API Definition

```
int MVID_CR_Algorithm_GetFloatValue(  
    void                *handle,  
    const char* const    strParamKeyName,  
    float* const         pfValue  
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to **Algorithm Parameters** .

pfValue

[IN][OUT] Parameter value

Return Value

Return 0 for success, and return **Error Code** for failure.

3.4.5 MVID_CR_Algorithm_SetStringValue

Set algorithm parameters of type string.

API Definition

```
int MVID_CR_Algorithm_SetStringValue(  
    void *handle,  
    const char* const strParamKeyName,  
    const char* const strValue  
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to [***Algorithm Parameters***](#).

pValue

[IN] Parameter value

Return Value

Return 0 for success, and return [***Error Code***](#) for failure.

3.4.6 MVID_CR_Algorithm_GetStringValue

Get algorithm parameters of type string.

API Definition

```
int MVID_CR_Algorithm_GetFloatValue(  
    void *handle,  
    const char* const strParamKeyName,  
    char* const strValue  
);
```

Parameters

handle

[IN] Handle.

strParamKeyName

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to [***Algorithm Parameters***](#).

strValue

[IN][OUT] Parameter value

Return Value

Return *0* for success, and return **Error Code** for failure.

Chapter 4 Data Structure and Enumeration

4.1 Data Structure

4.1.1 MVID_CAM_ENUMVALUE

Structure about values of type enum.

Member	Data Type	Description
nCurValue	unsigned int	Current value
nSupportedNum	unsigned int	The number of data records
nSupportValue	Array of unsigned int	Supported enumeration types, each array indicates one type, the maximum size is 64 bits.
nReserved	Array of unsigned int	Reserved.

4.1.2 MVID_CAM_FLOATVALUE

Structure about values of type float.

Member	Data Type	Description
fCurValue	float	Current value
fMax	float	The maximum value
fMin	float	The minimum value
nReserved	Array of unsigned int	Reserved.

4.1.3 MVID_CAM_INTVALUE_EX

Structure about values of type int64_t.

Member	Data Type	Description
nCurValue	int64_t	Current value
nMax	int64_t	The maximum value

Member	Data Type	Description
nMin	int64_t	The minimum value
nInc	int64_t	Increment
nReserved	Array of unsigned int	Reserved.

4.1.4 MVID_CAM_OUTPUT_INFO

Structure about output image information.

Member	Data Type	Description
stImage	<u>MVID_IMAGE_INFO</u>	Image information
stCodeList	<u>MVID_CODE_INFO_LIST</u>	Barcode information
plImageWaybill	unsigned char*	Image matting buffer
nImageWaybillLen	unsigned int	Image size
enWaybillImageType	<u>MVID_IMAGE_TYPE</u>	Image format
nReserved	Array of unsigned int	Reserved.

4.1.5 MVID_CAM_STRINGVALUE

Structure about values of type string.

Member	Data Type	Description
chCurValue	Array of char	Current value
nMaxLength	int64_t	The maximum size
nReserved	Array of unsigned int	Reserved.

4.1.6 MVID_CAMERA_INFO

Device information structure.

Member	Data Type	Description
nCamType	unsigned int	Camera type: USB, GigE
chManufacturerName	Array of unsigned char	Manufacturer name
chModelName	Array of unsigned char	Device model
chDeviceVersion	Array of unsigned char	Device version No.
chManufacturerSpecificInfo	Array of unsigned char	Manufacturer's Specific information
chSerialNumber	Array of unsigned char	Device serial No.
chUserDefinedName	Array of unsigned char	Custom name
nMacAddrHigh	Array of unsigned int	High MAC address
nMacAddrLow	unsigned int	Low MAC address
nCommomReaserved	Array of unsigned int	Reserved
nCurrentIp	unsigned int	Current IP address
nNetExport	unsigned int	IP Address of GigE camera
nGigEReserved	Array of unsigned int	Reserved for GigE camera
CrtlInEndPoint	unsigned char	Control input endpoint
CrtlOutEndPoint	unsigned char	Control output endpoint
StreamEndPoint	unsigned char	Stream endpoint
EventEndPoint	unsigned char	Event endpoint
idVendor	unsigned short	Vendor ID
idProduct	unsigned short	Product ID
nDeviceNumber	unsigned int	Device ID
nUsbReserved	Array of unsigned int	Reserved for USB camera
bSelectDevice	bool	Whether the camera belongs to the target model: true, false.
nReserved	Array of unsigned int	Reserved.

4.1.7 MVID_CAMERA_INFO_LIST

Device information list structure.

Member	Data Type	Description
nCamNum	unsigned int	The number of online devices
pstCamInfo	<u>MVID_CAMERA_INFO</u> *	Device information, up to 256 devices can be supported.

4.1.8 MVID_CODE_INFO

Bar code information structure.

Member	Data Type	Description
strCode	Array of unsigned char	Character, the maximum size is 4096
nLen	int	Character size
enBarType	<u>MVID_CODE_TYPE</u>	Barcode type
stCornerPt	Array of <u>MVID_POINT_I</u>	Barcode location
nAngle	int	Barcode angle, range: [0, 3600°]
nFilterFlag	int	Filter identifier: 0- normal code, 1-filter code
nReserved	Array of unsigned int	Reserved.

4.1.9 MVID_CODE_INFO_LIST

Structure about bar code information list.

Member	Data Type	Description
nCodeNum	int	The number of bar codes
stCodeInfo	Array of <u>MVID_CODE_INFO</u>	Bar code information, maximum size: 256
nReserved	Array of unsigned int	Reserved.

4.1.10 MVID_EVENT_OUT_INFO

Event callback information

Member	Data Type	Description
EventName	Array of char	Event name, , maximum size: 128
nEventID	unsigned short	Event ID
nStreamChannel	unsigned short	Stream channel ID
nBlockIdHigh	unsigned int	High-bit of frame ID
nBlockIdLow	unsigned int	Low-bit of frame ID
nTimestampHigh	unsigned int	High-bit of timestamp
nTimestampLow	unsigned int	Low-bit of timestamp
pEventData	void *	Event data
nEventDataSize	unsigned int	Event data size
nReserved	Array of unsigned int	Reserved.

4.1.11 MVID_IMAGE_INFO

Output image information structure.

Member	Data Type	Description
pImageBuf	unsigned char*	Original image buffer
nImageLen	unsigned int	Original image size
enImageType	<u>MVID_IMAGE_TYPE</u>	Image type
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
nFrameNum	unsigned int	Frame No.
nDevTimeStampHigh	unsigned int	Higher 32-bits of timestamp
nDevTimeStampLow	unsigned int	Lower 32-bits of timestamp
nReserved	Array of unsigned int	Reserved.

4.1.12 MVID_IMAGE_OUTPUT_MODE

Image output format structure.

Member	Macro Definition Value	Description
MVID_OUTPUT_NORMAL	0	Convert all non-mono8 images to BGR format
MVID_OUTPUT_RAW	1	Output in the original image format

4.1.13 MVID_POINT_I

Point coordinate information structure.

Member	Data Type	Description
nX	int	X-coordinate
nY	int	Y-coordinate

4.1.14 MVID_PROC_PARAM

Algorithm parameter information structure

Member	Data Type	Description
pImageBuf	unsigned char*	Original image buffer
nImageLen	unsigned int	Original image size
enImageType	<u>MVID_IMAGE_TYPE</u>	Image type
nWidth	unsigned short	Image width
nHeight	unsigned short	Image height
stCodeList	<u>MVID_CODE_INFO_LIST</u>	Barcode information
pImageWaybill	unsigned char*	Matting buffer, which is allocated by SDK
nImageWaybillLen	unsigned int	Image size
enWaybillImageType	<u>MVID_IMAGE_TYPE</u>	Image format
nReserved	Array of unsigned int	Reserved.

4.2 Enumeration

4.2.1 MVID_CODE_TYPE

Enumeration about bar code types.

Member	Macro Definition Value	Description
MVID_CODE_NONE	0	No recognizable bar code
MVID_CODE_TDCR_DM	1	DM code
MVID_CODE_TDCR_QR	2	QR code
MVID_CODE_NONE	8	EAN8 code
MVID_CODE_BCR_UPCE	9	UPCE code
MVID_CODE_BCR_UPCA	12	UPCA code
MVID_CODE_BCR_EAN13	13	EAN13 code
MVID_CODE_BCR_ISBN13	14	ISBN13 code
MVID_CODE_BCR_CODABAR	20	Codabar code
MVID_CODE_BCR_ITF25	25	ITF25 code
MVID_CODE_BCR_CODE39	39	Code 39
MVID_CODE_BCR_CODE93	93	Code 93
MVID_CODE_BCR_CODE128	128	Code 128

4.2.2 MVID_IMAGE_TYPE

Picture saving format enumeration.

Member	Macro Definition Value
MVID_IMAGE_Undefined	0
MVID_IMAGE_MONO8	1
MVID_IMAGE_JPEG	2
MVID_IMAGE_BMP	3
MVID_IMAGE_RGB24	4

Member	Macro Definition Value
MVID_IMAGE_BGR24	5
MVID_IMAGE_MONO10	6
MVID_IMAGE_MONO10_Packed	7
MVID_IMAGE_MONO12	8
MVID_IMAGE_MONO12_Packed	9
MVID_IMAGE_MONO16	10
MVID_IMAGE_BayerGR8	11
MVID_IMAGE_BayerRG8	12
MVID_IMAGE_BayerGB8	13
MVID_IMAGE_BayerBG8	14
MVID_IMAGE_BayerGR10	15
MVID_IMAGE_BayerRG10	16
MVID_IMAGE_BayerGB10	17
MVID_IMAGE_BayerBG10	18
MVID_IMAGE_BayerGR12	19
MVID_IMAGE_BayerRG12	20
MVID_IMAGE_BayerGB12	21
MVID_IMAGE_BayerBG12	22
MVID_IMAGE_BayerGR10_Packed	23
MVID_IMAGE_BayerRG10_Packed	24
MVID_IMAGE_BayerGB10_Packed	25
MVID_IMAGE_BayerBG10_Packed	26
MVID_IMAGE_BayerGR12_Packed	27
MVID_IMAGE_BayerRG12_Packed	28
MVID_IMAGE_BayerGB12_Packed	29
MVID_IMAGE_BayerBG12_Packed	30
MVID_IMAGE_YUV422_Packed	31
MVID_IMAGE_YUV422_YUYV_Packed	32

Member	Macro Definition Value
MVID_IMAGE_RGB8_Packed	33
MVID_IMAGE_BGR8_Packed	34
MVID_IMAGE_RGBA8_Packed	35
MVID_IMAGE_BGRA8_Packed	36

4.2.3 MVID_IMAGE_OUTPUT_MODE

Image output format structure.

Member	Macro Definition Value	Description
MVID_OUTPUT_NORMAL	0	Convert all non-mono8 images to BGR format
MVID_OUTPUT_RAW	1	Output in the original image format

Appendix A. Appendixes

A.1 Algorithm Parameters

This page provides the description and value setting suggestions of algorithm parameters that are obtained from Machine Vision Software according to camera type.

One-Dimension Parameters

Parameter Name	Description	Value Range
KEY_BCR_ABILITY	Algorithm capability set, including Code39[1], Code128[2], CodeBar[4], EAN[8], ITF25[16], CODE93[32]	Range: [0, 63], default: 63
KEY_BCR_ROI_X	ROI offset in X direction	Range: [0, 65535], default: 0
KEY_BCR_ROI_Y	ROI offset in Y direction	Range: [0, 65535], default: 0
KEY_BCR_ROI_WIDTH	ROI width	Range: [100, 65535], default: 65535
KEY_BCR_ROI_HEIGHT	ROI height	Range: [40, 65535], default: 65535
KEY_BCR_MAX_WIDTH	The maximum width of images that can be recognized by the algorithm.	Range: [0, 65535], default: 3840
KEY_BCR_MAX_HEIGHT	The maximum height of images that can be recognized by the algorithm.	Range: [0, 65535], default: 2748
KEY_BCR_LOCBARNUM	The number of barcodes to read	Range: [1, 200], default: 4
KEY_BCR_LOCWINSIZE	Barcode height	Range: [4, 65], default: 4
KEY_BCR_WAITINGTIME	Algorithm library's maximum running time. The current image processing will be forced to stop when the specific time runs out.	Range: [0, 5000], default: 500
KEY_BCR_SEGQUIETW	Width of the barcode quiet zone	Range: [0, 200], default: 30

Parameter Name	Description	Value Range
KEY_BCR_DFKSIZELOWERLIMIT	Lower-limit of barcode width, below which the bar code will be filtered out.	Range: [0, 4000], default: 30
KEY_BCR_DFKSIZEUPPERLIMIT	Upper-limit of barcode width, above which the barcode will be filtered out.	Range: [0, 4000], default: 2400
KEY_BCR_SAVEIMAGELEVEL	Sensitivity of saving unanalyzed images	Range: [1, 3], default: 1
KEY_BCR_APPMODE	Algorithm library operating mode (expert mode or normal mode)	Range: [0, 2147483646], default: 1
KEY_BCR_DISTORTION	Whether to enable perspective distortion switch in expert mode	Range: [0, 2147483646], default: 0
KEY_BCR_WHITEGAP	Whether to enable printing quality switch in expert mode	Range: [0, 2147483646], default: 0
KEY_BCR_SPOT	Whether to enable specular reflective switch in expert mode	Range: [0, 2147483646], default: 0
KEY_BCR_SAMPLELEVEL	Image sampling scale	Range: [1, 8], default: 1
KEY_BCR_IAMGEMORPH	Image morphology preprocessing	Range: [0, 2], default: 0
KEY_BCR_DELERRFLAG	Whether to enable reducing the error identification	Range: [0, 1], default: 1

Two-Dimension Parameters

Parameter Name	Description	Value Range
KEY_TDCR_ABILITY	Algorithm capability set, including QR[1], DM[2]	Range: [0, 3], default: 3
KEY_TDCR_ROI_X	ROI offset in X direction	Range: [0, 65535], default: 0
KEY_TDCR_ROI_Y	ROI offset in Y direction	Range: [0, 65535], default: 0
KEY_TDCR_ROI_WIDTH	ROI width	Range: [128, 65535], default: 65535
KEY_TDCR_ROI_HEIGHT	ROI height	Range: [128, 65535], default: 65535

Parameter Name	Description	Value Range
KEY_TDCR_MAX_WIDTH	The maximum width of images that can be recognized by the algorithm.	Range: [0, 65535], default: 3840
KEY_TDCR_MAX_HEIGHT	The maximum height of images that can be recognized by the algorithm.	Range: [0, 65535], default: 2748
KEY_TDCR_LOCCODENUM	The number of ROIs outputted by detection module.	Range: [1, 1000], default: 5
KEY_TDCR_MINBARSIZE	The minimum width and height of barcodes when filtering blobs	Range: [20, 1000], default: 40
KEY_TDCR_MAXBARSIZE	The maximum width and height when filtering blobs	Range: [20, 1000], default: 300
KEY_TDCR_MIRRORMODE	Whether to enable mirror mode	Range: [0, 2], default: 0
KEY_TDCR_SAMPLELEVEL	Image downsampling ratio	Range: [1, 8], default: 1
KEY_TDCR_CODECOLOR	Identifier of the black bar code on white background	Range: [0, 2], default: 0
KEY_TDCR_DISCRETEFLAG	Code flag: "0"-continuous code, "1"-discrete code, "2"-self-adaptive	Range: [0, 2], default: 0
KEY_TDCR_DISTORTIONFLAG	QR distortion parameter	Range: [0, 1], default: 0
KEY_TDCR_WAITINGTIME	Exiting timeout	Range: [0, 5000], default: 1000
KEY_TDCR_DEBUGFLAG	Whether to display debug information	Range: [0, 1], default: 0
KEY_TDCR_ADVANCEPARAM	Advanced parameters	Range: [0, 2147483640], default: 0
KEY_TDCR_ADVANCEPARAM2	Advanced parameters 2	Range: [0, 2147483640], default: 0
KEY_TDCR_RECTANGLE	Code types: 0-square, 1-rectangle, 2-compatible mode	Range: [0, 2], default: 0
KEY_TDCR_APPMODE	Algorithm library operating mode (expert mode, normal mode, or speed mode)	Range: [0, 2], default: 0

Image Matting Parameters

Parameter Name	Description	Value Range
KEY_WAYBILL_ABILITY	Algorithm capability set	Range: [1, 7], default: 7
KEY_WAYBILL_MAX_WIDTH	The maximum width of images that can be recognized by the algorithm.	Range: [0, 65535], default: 3840
KEY_WAYBILL_MAX_HEIGHT	The maximum height of images that can be recognized by the algorithm.	Range: [0, 65535], default: 2748
KEY_WAYBILL_OUTPUTIMAGETYPE	Image output format	Range: [1, 3], default: 2 "1"-mono8, "2"-JPG, "3"-BMP
KEY_WAYBILL_JPGQUALITY	JPG encoding quality	Range: [1, 100], default: 80
KEY_WAYBILL_MINWIDTH	Minimum width of waybill	Range: [15, 2592], default: 100
KEY_WAYBILL_MINHEIGHT	Minimum height of waybill	Range: [10, 2048], default: 100
KEY_WAYBILL_MAXWIDTH	Maximum width of waybill	Range: [15, 3072], default: 3072
KEY_WAYBILL_MAXHEIGHT	Maximum height of waybill	Range: [10, 2048], default: 2048
KEY_WAYBILL_MORPHTIMES	Expansion times	Range: [0, 10], default: 0
KEY_WAYBILL_GRAYLOW	Minimum gray value of the bar code and character gray on the waybill	Range: [0, 255], default: 0
KEY_WAYBILL_GRAYMID	Median gray value of waybill which is used to distinguish barcode from background	Range: [0, 255], default: 70
KEY_WAYBILL_GRAYHIGH	Maximum gray value of waybill background	Range: [0, 255], default: 130
KEY_WAYBILL_BINARYADAPTIVE	Adaptive binarization	Range: [0, 1], default: 1
KEY_WAYBILL_BOUNDARYROW	Expand the edge in row direction when matting waybill	Range: [0, 2000], default: 0
KEY_WAYBILL_BOUNDARYCOL	Expand the edge in column direction when matting waybill	Range: [0, 2000], default: 0

Parameter Name	Description	Value Range
KEY_WAYBILL_MAXBILLBARHEIGTHRATIO	Maximum height ratio of waybill to barcode	Range: [1, 100], default: 20
KEY_WAYBILL_MAXBILLBARWIDTHRATIO	Maximum width ratio of waybill to barcode	Range: [1, 100], default: 5
KEY_WAYBILL_MINBILLBARHEIGTHRATIO	Minimum height ratio of waybill to barcode	Range: [1, 100], default: 5
KEY_WAYBILL_MINBILLBARWIDTHRATIO	Minimum width ratio of waybill to barcode	Range: [1, 100], default: 2
KEY_WAYBILL_ENHANCEMETHOD	Enhancement method	Range: [1, 4], default: 2
KEY_WAYBILL_ENHANCECLIPRATIOLOW	Increase the low threshold ratio of stretching	Range: [0, 100], default: 1
KEY_WAYBILL_ENHANCECLIPRATIOHIGH	Increase the high threshold ratio of stretching	Range: [0, 100], default: 99
KEY_WAYBILL_ENHANCECONTRASTFACTOR	Contrast ratio	Range: [1, 10000], default: 100
KEY_WAYBILL_ENHANCESHARPENFACTOR	Sharpness	Range: [0,10000], default: 0
KEY_WAYBILL_SHARPENKERNELSIZE	Size of sharpening filter core	Range: [3, 15], default: 3
KEY_WAYBILL_CODEBOUNDARYROW	Expand the edge in row direction when matting weight memo	Range: [0, 2000], default: 0
KEY_WAYBILL_CODEBOUNDARYCOL	Expand the edge in column direction when matting weight memo	Range: [0, 2000], default: 0

A.2 Error Code

You can search for the detailed error descriptions according to returned error codes or error names.

Error Code	Error Name	Description
0x00000000	MVID_CR_OK	Succeeded.
0x80000000	MVID_CR_E_HANDLE	Error or invalid handle
0x80000001	MVID_CR_E_SUPPORT	The function is not supported
0x80000002	MVID_CR_E_BUFOVER	Buffer is full
0x80000003	MVID_CR_E_CALLORDER	Incorrect calling order
0x80000004	MVID_CR_E_PARAMETER	Incorrect parameter
0x80000005	MVID_CR_E_RESOURCE	Applying resource failed.
0x80000006	MVID_CR_E_NODATA	No data
0x80000007	MVID_CR_E_PRECONDITION	Incorrect precondition, or the running environment changed.
0x80000008	MVID_CR_E_ENCRYPT	Credential error, possibly because the dongle was not installed or expired.
0x8000000A	MVID_CR_E_RULE	Filter rule error.
0x8000000B	MVID_CR_E_LOAD_LIBRARY	Dynamically importing the DLL file failed.
0x80000012	MVID_CR_E_JPGENC	JPG encoding error.
0x80000013	MVID_CR_E_IMAGE	Abnormal image. Incomplete image caused by packet loss or incorrect image format, width, or height.
0x80000014	MVID_CR_E_CONVERT	Format conversion error
0x800000FF	MVID_CR_E_UNKNOW	Unknown error
0x80000100	MVID_CR_E_GC_GENERIC	Generic error
0x80000101	MVID_CR_E_GC_ARGUMENT	Invalid parameter
0x80000102	MVID_CR_E_GC_RANGE	The value is out of range.
0x80000103	MVID_CR_E_GC_PROPERTY	Attribute error
0x80000104	MVID_CR_E_GC_RUNTIME	Running environment error
0x80000105	MVID_CR_E_GC_LOGICAL	Incorrect logic
0x80000106	MVID_CR_E_GC_ACCESS	Node accessing condition error
0x80000107	MVID_CR_E_GC_TIMEOUT	Timed out

Error Code	Error Name	Description
0x80000108	MVID_CR_E_GC_DYNAMICCAST	Conversion exception
0x800001FF	MVID_CR_E_GC_UNKNOW	GenICam unknown error
0x80000200	MVID_CR_E_NOT_IMPLEMENTED	Command is not supported by device.
0x80000201	MVID_CR_E_INVALID_ADDRESS	Target address does not exist
0x80000202	MVID_CR_E_WRITE_PROTECT	Target address is not writable.
0x80000203	MVID_CR_E_ACCESS_DENIED	No access permission
0x80000204	MVID_CR_E_BUSY	Device is busy, or network is disconnected.
0x80000205	MVID_CR_E_PACKET	Network packet error
0x80000206	MVID_CR_E_NETER	Network error
0x80000221	MVID_CR_E_IP_CONFLICT	IP address conflicted.
0x80000300	MVID_CR_E_USB_READ	USB read error
0x80000301	MVID_CR_E_USB_WRITE	USB write error
0x80000302	MVID_CR_E_USB_DEVICE	Device exception
0x80000303	MVID_CR_E_USB_GENICAM	GenICam error
0x80000304	MVID_CR_E_USB_BANDWIDTH	Insufficient bandwidth
0x80000305	MVID_CR_E_USB_DRIVER	Driver is mismatched, or is not installed.
0x800003FF	MVID_CR_E_USB_UNKNOW	USB unknown error
0x80002100	MVID_CR_E_CAMERA	Camera error
0x80002200	MVID_CR_E_BCR	1D barcode error
0x80002300	MVID_CR_E_TDCR	2D barcode error
0x80002400	MVID_CR_E_WAYBILL	Matting error
0x80002500	MVID_CR_E_SCRIPT	Script rule error

